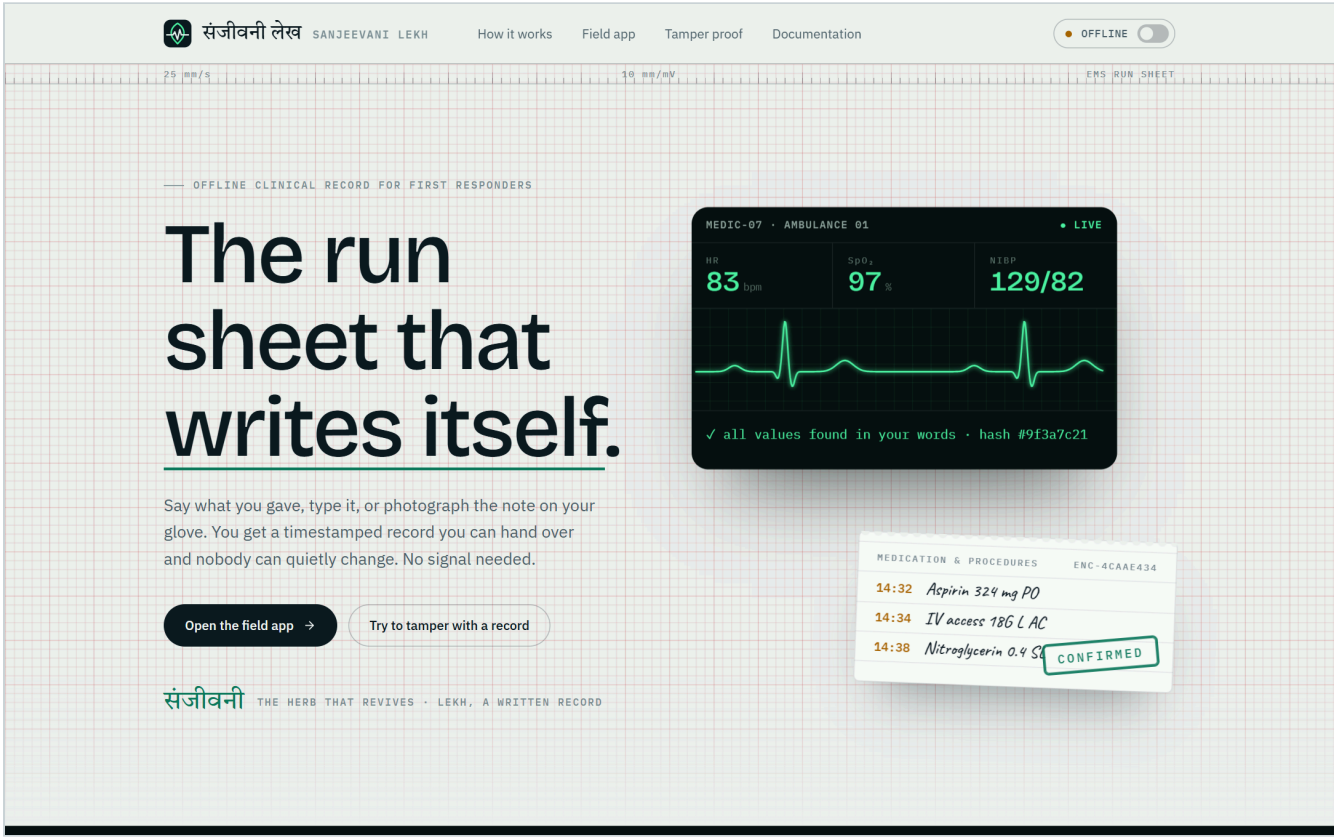
SANJEEVANI LEKH

Offline-first clinical record for first responders

System design and project guide. How a spoken, typed or photographed field note becomes a verified, tamper-evident record and a printable handover.



PROJECT	TEAM	VERSION
Virtusa Jatayu 2026	Milind Verma, Aditya Bipin Kulkarni, Gaurav Kumar, Nikhil Agarwal	October 2026

Research and portfolio project. Not a medical device. Every record is confirmed by a responder.

CONTENTS

1 Overview
2 System architecture
3 From note to record
4 Online and offline modes
5 Audit chain and data model
6 End-to-end sequence
7 User interface
8 API reference
9 Testing and measured results
10 Running the project
11 Limitations and next steps

1 Overview

The problem

In an emergency, responders give drugs and perform procedures quickly and have no time to write them down. Notes end up on gloves, scraps of paper, or in memory. That causes three problems: the receiving hospital may not know a drug has already been given, doses and times written later from memory are often wrong, and nobody can later show exactly what happened and when.

What the system does

A responder speaks, types, or photographs a field note. Sanjeevani Lekh extracts every medication and procedure into a structured record, checks each value against the responder's own words, asks the responder to resolve anything uncertain, and then seals the confirmed record into an append-only log in which each record carries the hash of the one before it. From that log it prints a one-page handover for the receiving clinician.

Design principles

- **Never guess.** A value must be traceable to words in the original note. If it is not, it is left empty and flagged, not filled in.
- **A human confirms every record.** Items with a review flag must be accepted or removed before the record can be sealed.
- **Tamper-evident, not just stored.** Records are hash-chained, and the database itself refuses updates and deletes.
- **Offline first, cloud optional.** The whole flow works on the device. An optional online mode uses Groq, and any cloud failure falls back to the local models.
- **Always produce a draft.** If the cloud and the local model are both unavailable, a built-in rule-based extractor still returns something the responder can review.

16	10	17	85
API endpoints	validation rules, plain Python	drugs in the reference list	automated tests passing

PART	WHAT IT IS
Backend	Python and FastAPI service with an SQLite audit log. About 3,900 lines in `backend/app` and 1,700 lines of tests.
Responder app	React and Vite single-page app: landing page, field app, documentation. Served by the backend. About 6,600 lines.
Supervisor dashboard	Next.js, read-only, reads the synced `audit_log` table from Supabase. About 1,300 lines.
Output	FHIR-lite `MedicationAdministration` and `Procedure` resources in a `Bundle`, plus an HTML and PDF handover report.

2 System architecture

Figure 1 on the next page shows the whole system. The tables below name each part.

2.1 Backend modules

MODULE	RESPONSIBILITY
`main.py`	FastAPI app and every route. Runs the draft pipeline, serves the built UI, restricts CORS to localhost.
`config.py`	Settings loaded from `backend/.env`. Blank values fall back to defaults.
`transcribe.py`	Voice to text with whisper.cpp when its binary and model exist, otherwise a local Whisper pipeline. Uses a medical vocabulary hint prompt.
`ocr.py`	Photo to text with TrOCR (`microsoft/trocr-base-handwritten`) and EasyOCR for lines. Reports per-line confidence and flags low-confidence lines.
`extract.py`	Local Ollama extraction constrained by a JSON schema, one retry on invalid output. Also writes the short summary shown on the review screen.
`cloud.py`	Groq client for extraction, summary and hosted Whisper. Raises one error type on any failure so the caller can fall back.
`rule_extractor.py`	Built-in extractor used when no model is reachable.
`normalise.py`	Spoken numbers ("point four" to 0.4), times, routes ("under the tongue" to SL), units and drug aliases (nitro to nitroglycerin, using `data/drug_reference.json`).
`validate.py`	The 10 safety rules. No AI involved. Produces flags.
`fhir.py`	Maps items to FHIR-lite resources with the source words attached as an extension.
`clinical_ner.py`	Optional cross-check with a clinical NER model and Bio_ClinicalBERT embeddings.
`db.py`	The only module that touches SQLite. Hash chaining, triggers, verification.
`report.py`	Handover report as HTML (Jinja2) and PDF (ReportLab).
`sync.py`	Pushes unsynced rows to Supabase and marks them synced.

2.2 Technology

LAYER	TECHNOLOGY
Backend	Python, FastAPI, Pydantic, httpx, SQLite (WAL mode), Jinja2, ReportLab
Speech	whisper.cpp (`ggml-small.en`), or Whisper via transformers; Groq `whisper-large-v3-turbo` online
Handwriting	TrOCR and EasyOCR
Language model	Ollama (default `qwen3:8b`) offline; Groq `openai/gpt-oss-120b` online
Clinical NER	`samrawal/bert-base-uncased_clinical-ner` and `emilyalsentzer/Bio_ClinicalBERT`
Responder app	React 18, Vite, Lenis smooth scroll, lucide icons
Dashboard	Next.js 14, Supabase client

Figure 1 System architecture

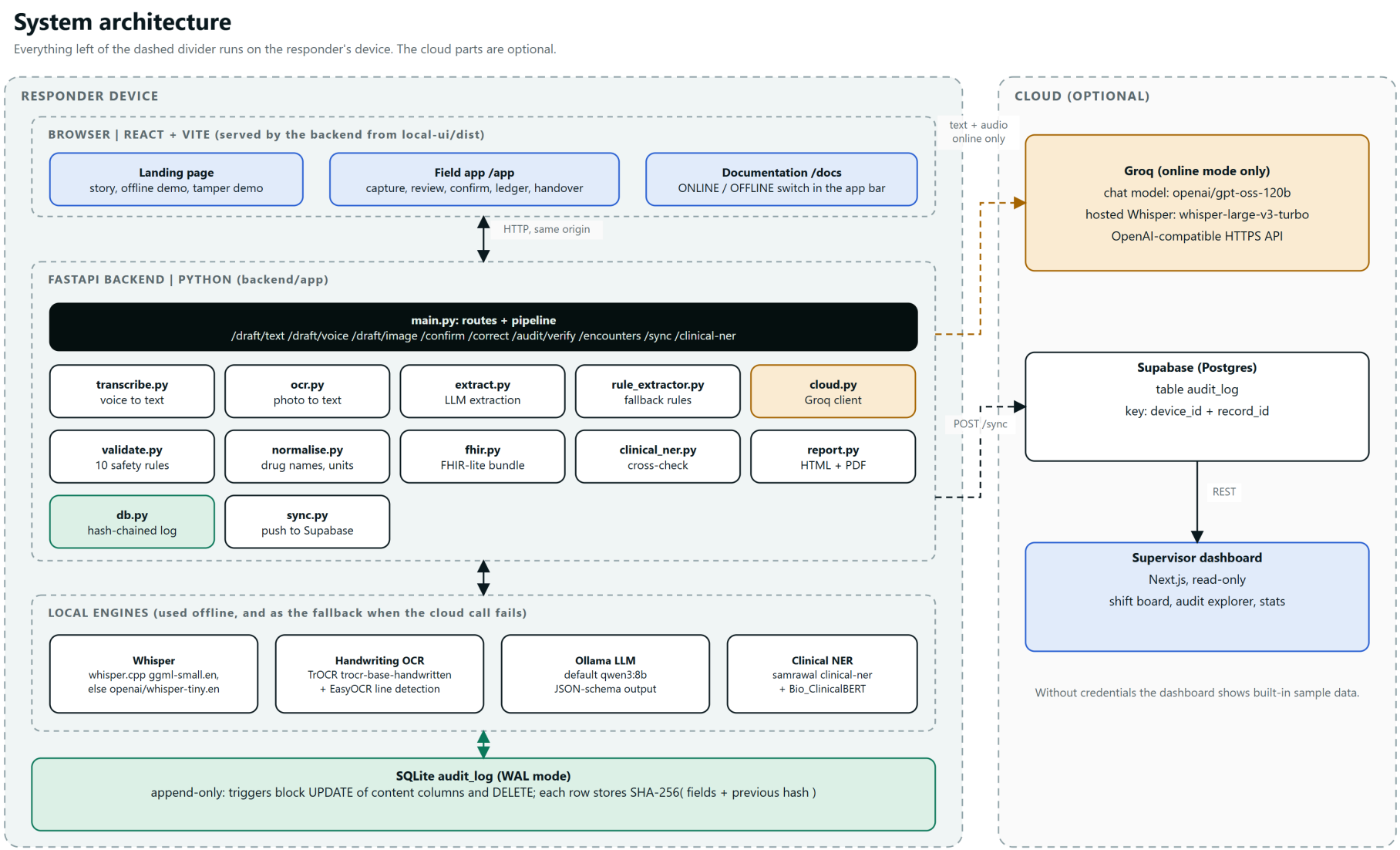


Figure 1. Everything on the left runs on the responder's device. Groq and Supabase are optional.

3 From note to record

From spoken note to sealed record

Each step either passes a value on or raises a flag. Nothing is silently guessed.

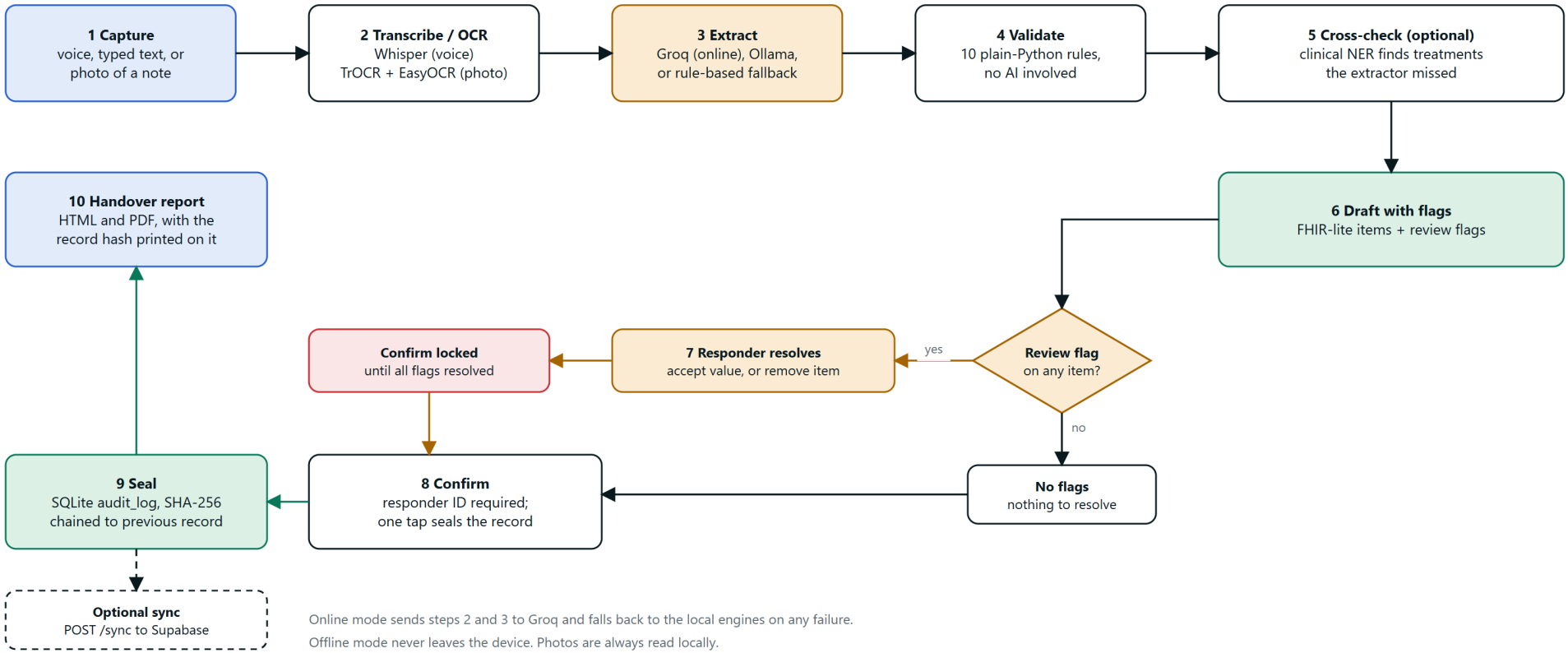


Figure 2. Each step either passes a value on or raises a flag.

3.1 Steps in detail

STEP	WHAT HAPPENS
Capture	Voice goes to `POST /draft/voice`, typed text to `/draft/text`, a photo to `/draft/image`. Uploaded audio and images are deleted from disk after processing.
Extract	A model returns `medications` and `procedures` as JSON that must match a schema. The prompt (`prompts/extract_v1.txt`) requires that every item copy its exact words into `source_text`. Temperature is 0. One retry with a stricter prompt if the output does not validate.
Normalise	Spoken numbers, times, routes, units and brand or shorthand drug names are mapped to standard forms.
Validate	Ten plain-Python rules (table below) compare every item with the original note and the drug reference list.
Cross-check	Optional. The NER model tags treatments, problems and tests. A treatment in the note that was not extracted raises an info flag.
Review	The responder sees the original words next to the extracted items. Items with a review flag show the reason and must be accepted or removed.
Confirm and seal	A responder ID is required. The record is appended to the audit log (section 5).
Report	One-page handover as HTML or PDF with the record hash printed on it.

3.2 Extracted data

ITEM	FIELDS
Medication	`time`, `drug`, `dose_value`, `dose_unit`, `route`, `source_text`, `indication` (looked up from a built-in table; a local-model query only for unlisted drugs)
Procedure	`time`, `name`, `details`, `source_text`

Items become FHIR-lite resources in a `Bundle` of type `collection`: `MedicationAdministration` for medications and `Procedure` for procedures, each carrying the responder's own words as an extension (`source_text`).

3.3 Validation rules

FLAG CODE	SEVERITY	RAISED WHEN
`source_text_not_found`	needs review	The item's source words do not appear in the note.
`missing_dose`	needs review	A medication has no dose.
`missing_unit`	needs review	A dose was given but no unit was stated.
`unit_inferred`	needs review	The extracted unit is not mentioned in the item's own source words.
`missing_route`	needs review	No route was stated.
`missing_time`	info	No time was stated for the item.
`unknown_drug`	needs review	The drug is not in the 17-drug reference list.
`dose_out_of_range`	needs review	The dose is outside the reference range for that drug.
`spoken_correction`	needs review	The note contains a correction phrase such as "sorry" or "I mean", so the real dose needs a human check.
`duplicate_item`	needs review	The same medication administration appears twice.

Reference list: aspirin, nitroglycerin, epinephrine, naloxone, ondansetron, oxygen, albuterol, ipratropium, dextrose, glucagon, midazolam, fentanyl, morphine, amiodarone, adenosine, atropine, diphenhydramine.

4 Online and offline modes

Online and offline decision flow

The backend always produces a draft: cloud first when allowed, then the local model, then plain rules.

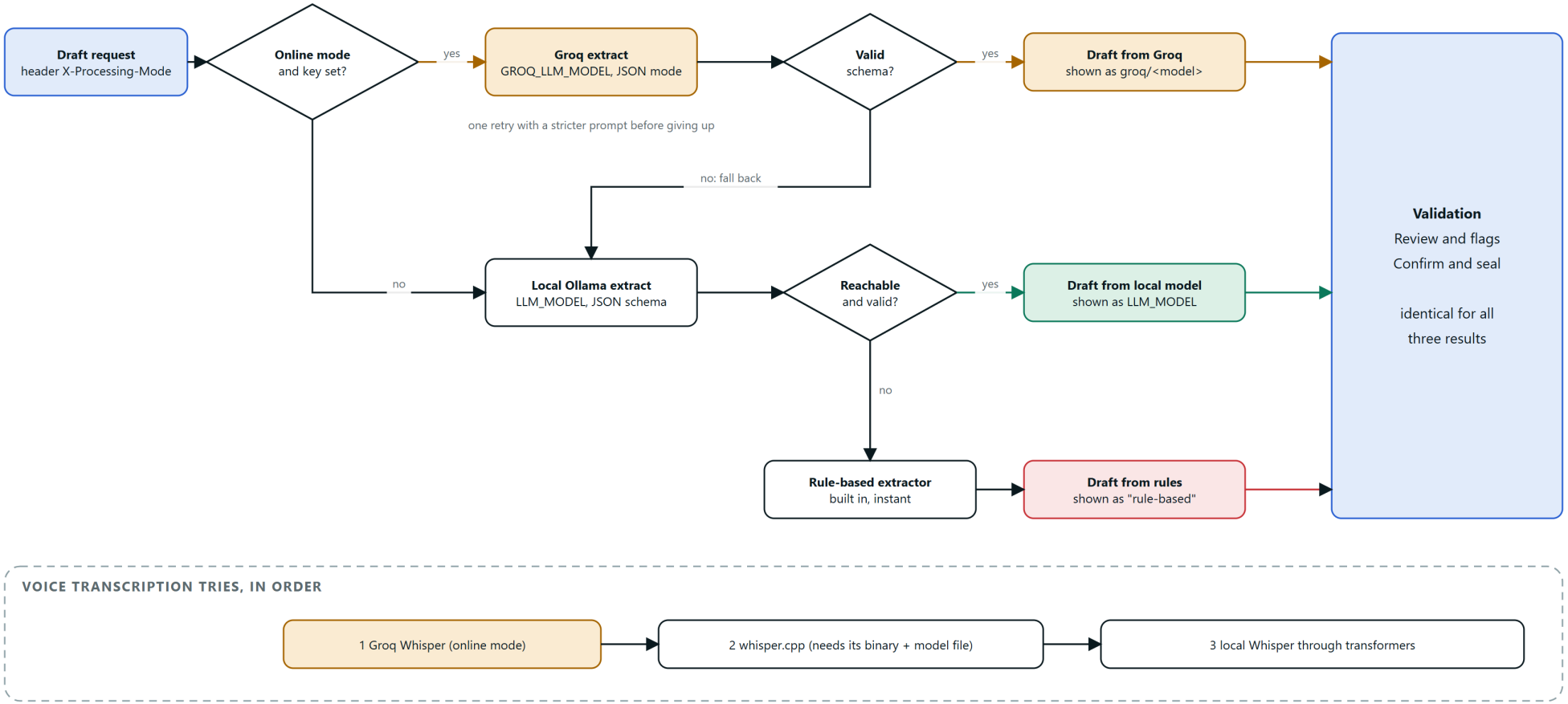


Figure 3. The backend always produces a draft: cloud first when allowed, then the local model, then plain rules.

4.1 How the mode is chosen

The field app has an ONLINE / OFFLINE switch in its top bar. It is sent with each request as the header `X-Processing-Mode`. Without the header the server default applies: `PROCESSING_MODE=auto` goes online when a Groq key is configured, and `offline` never contacts the cloud. Without a key the backend is always offline. In online mode any cloud failure (network, rate limit, invalid output after one retry) falls back to the local model, and then to the rule-based extractor. The engine that produced a draft is shown on the review screen and stored in the sealed record.

SETTING	DEFAULT	PURPOSE
`PROCESSING_MODE`	`auto`	`auto` or `offline`
`GROQ_API_KEY`	empty	Enables online mode
`GROQ_LLM_MODEL`	`openai/gpt-oss-120b`	Online extraction and summary
`GROQ_WHISPER_MODEL`	`whisper-large-v3-turbo`	Online voice transcription
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Local model server
`LLM_MODEL`	`qwen3:8b`	Local extraction model
`ENABLE_CLINICAL_NER`	`true`	Optional cross-check; needs torch and transformers

Privacy. In online mode the note text and recorded audio are sent to Groq, a third-party service. Use offline mode, or only synthetic data, wherever that is not acceptable. Photos are always read locally. Only the extraction and transcription steps use the cloud; validation, review and the audit log are always local.

4.2 Measured extraction quality

The repository includes 20 synthetic EMS notes with answer keys (`test-data`) and an evaluation script (`backend/scripts/evaluate.py`). It was run unmodified against two engines, using the same validation rules.

METRIC	RULE-BASED EXTRACTOR	GROQ `OPENAI/GPT-OSS-120B`
Medications found (of 27)	24, with 6 extra	27, with 0 extra
Procedures found (of 15)	10, with 7 extra	13, with 1 extra
Drug name F1	0.84	1.00
Dose exact match (value and unit)	78.3%	96.2%
Route F1	0.96	1.00
Time accuracy	100%	100%
Procedure F1	0.63	0.90
Wrong items caught by a review flag	7 of 13	0 of 1
Average extraction time	about 0.1 s	about 1.4 s

Caveats: 20 notes is a small sample written by the team, and the rule-based extractor may have been tuned on these same notes. The local Ollama model was not measured because Ollama is not installed on the machine used for this run. Two Groq calls needed a retry because of rate limiting. Speech and handwriting accuracy were not measured.

5 Audit chain and data model

Tamper-evident audit chain

Each record carries the hash of the one before it, so changing any earlier record breaks every record after it.

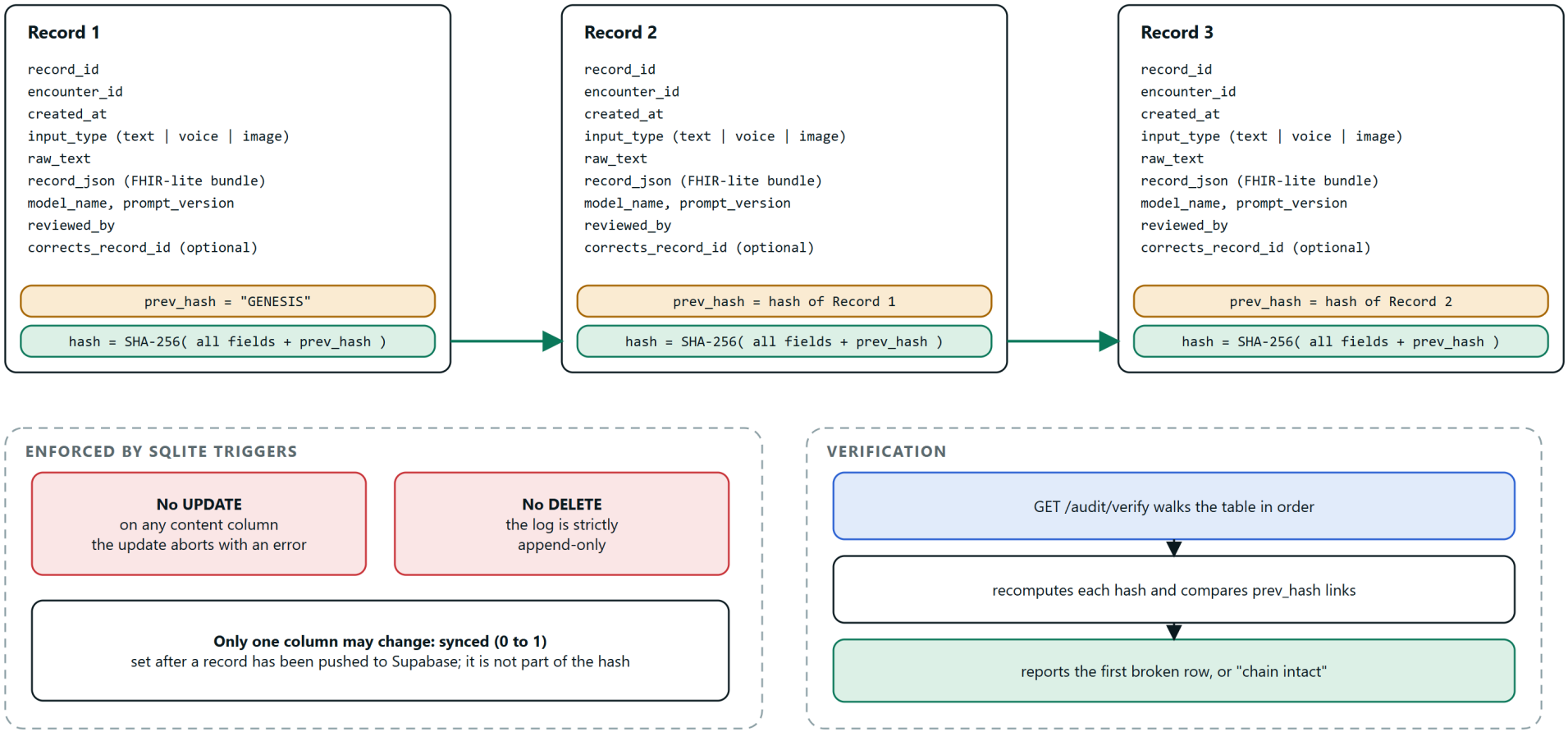


Figure 4. Changing any earlier record breaks the hash of every record after it.

5.1 The audit_log table

COLUMN	MEANING
`record_id`	Unique ID of the record, for example `REC-CB7DCF11-01`
`encounter_id`	The encounter the record belongs to; an encounter can hold several records
`created_at`	Timestamp of sealing
`input_type`	`text`, `voice` or `image`
`raw_text`	The responder's words as captured
`record_json`	The confirmed FHIR-lite bundle
`model_name`, `prompt_version`	Which engine and prompt produced the draft
`reviewed_by`	Responder ID that confirmed the record
`corrects_record_id`	Set when this record amends an earlier one; the earlier record is never edited
`prev_hash`, `hash`	The chain. The first record uses `GENESIS`
`synced`	0 or 1. The only column that may change, and it is not part of the hash

5.2 Guarantees and their limits

- `hash` is the SHA-256 of a canonical JSON (sorted keys) of all fields above plus `prev_hash`.
- SQLite triggers abort any UPDATE to a content column and any DELETE, so the application cannot rewrite history even by mistake.
- `GET /audit/verify` recomputes every hash in order and reports the first row that does not match.
- Corrections are new records that point back to the one they amend.

> **What this does and does not prove.** The chain makes tampering detectable. Someone with direct write access to the database file could rewrite the whole chain consistently, so the latest hash is only as trustworthy as the place it is kept. Printing it on the handover form is one way to anchor it outside the device.

5.3 Cloud sync and the dashboard

When a connection exists, `POST /sync` sends rows where `synced = 0` to the Supabase `audit_log` table and then marks them synced. Rows are keyed by `(device_id, record_id)` and merged on conflict, so retrying never creates duplicates. The schema is in `dashboard/supabase_schema.sql`. The Next.js dashboard only reads: a shift board with one lane per vehicle, counts, a 14-day sparkline, an encounter table, an audit explorer and a quality page. Without Supabase credentials it shows built-in sample data and says so in its header.

6 End-to-end sequence

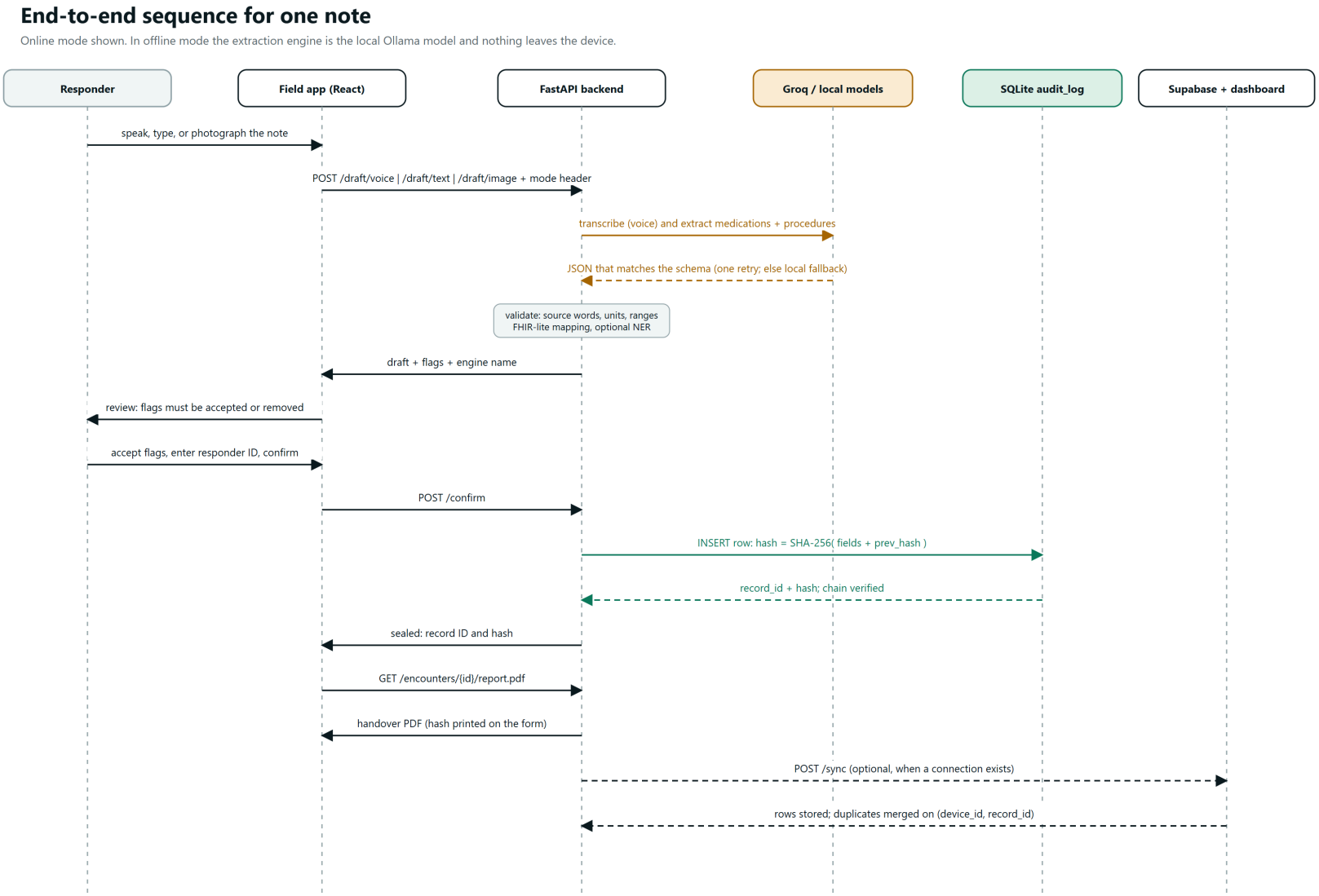
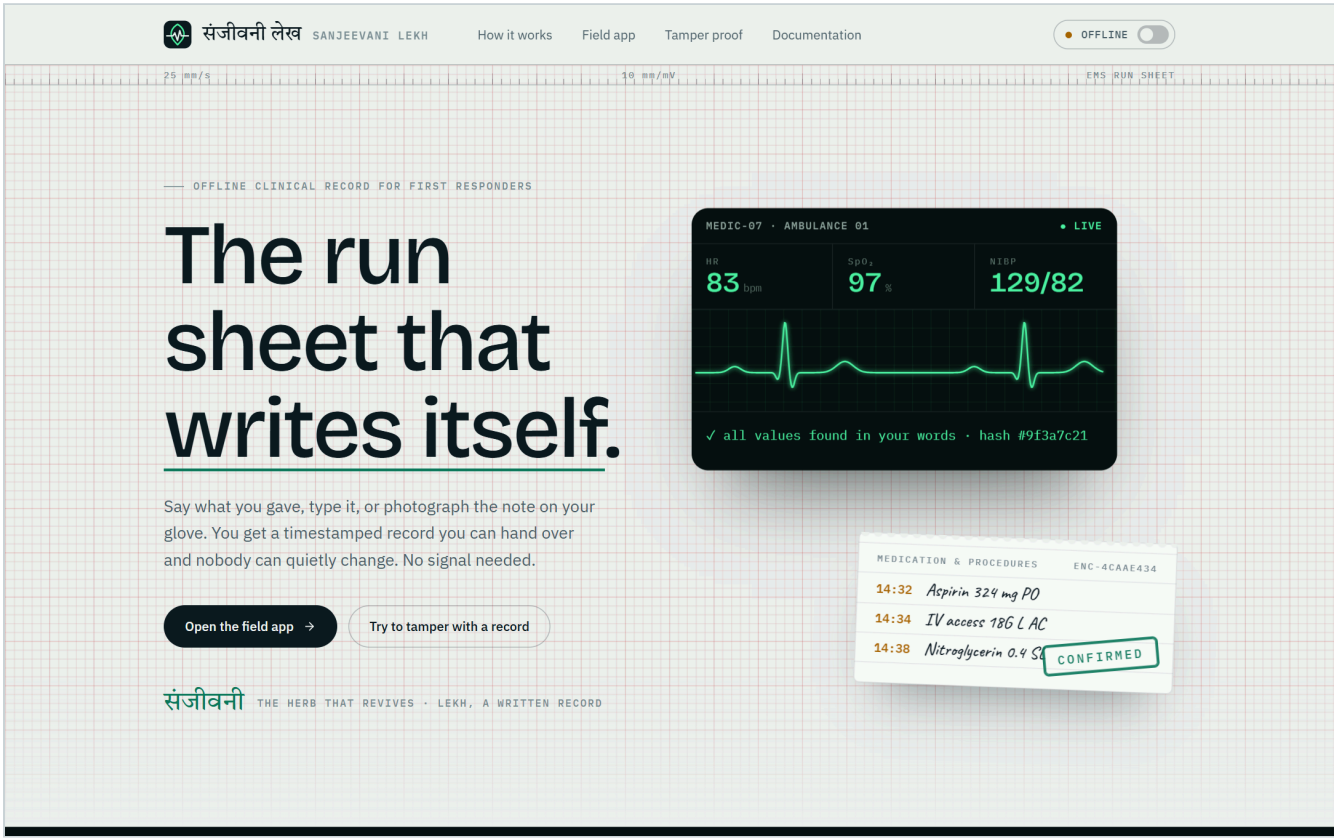


Figure 5. One note from capture to handover, online mode shown.

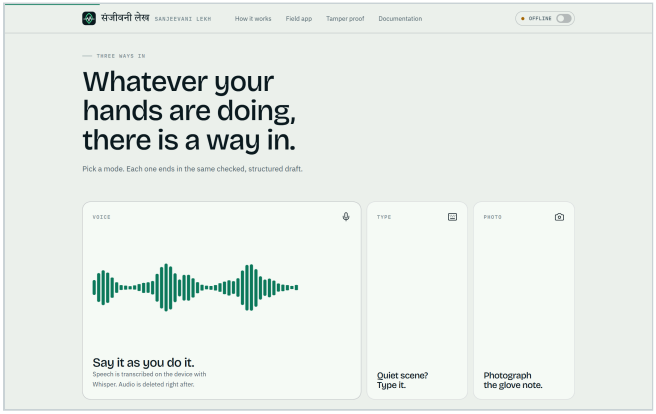
7 User interface

The interface uses one identity across the landing page, the field app, the documentation and the dashboard: ECG-paper backgrounds, ink text, herb green for "verified" and amber for "needs a person". Dark monitor-style panels are used only where something is live. Type is Bricolage Grotesque for headings, IBM Plex Sans and Mono for text and data, and Tiro Devanagari for the wordmark संजीवनी लेख. The logo is a leaf (the Sanjeevani herb) with a heartbeat through it.

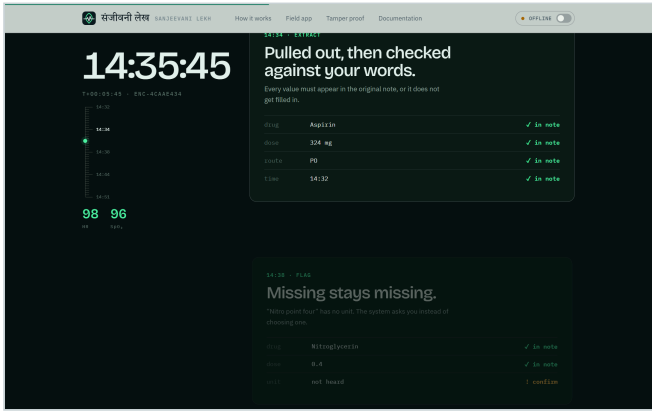
7.1 Landing page



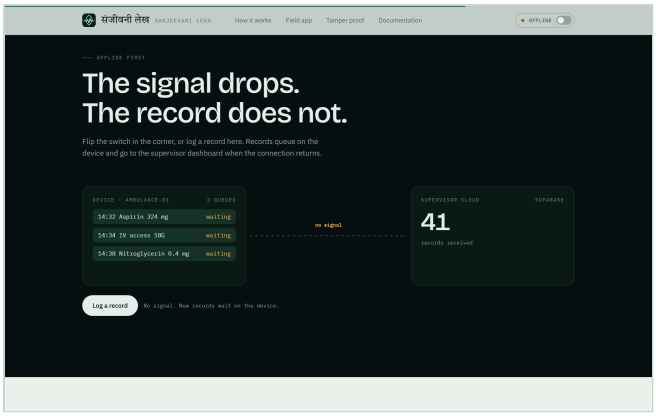
Hero: a live monitor and a run sheet that fills as the example note is dictated. The OFFLINE switch in the top bar simulates the connection for the sync demo.



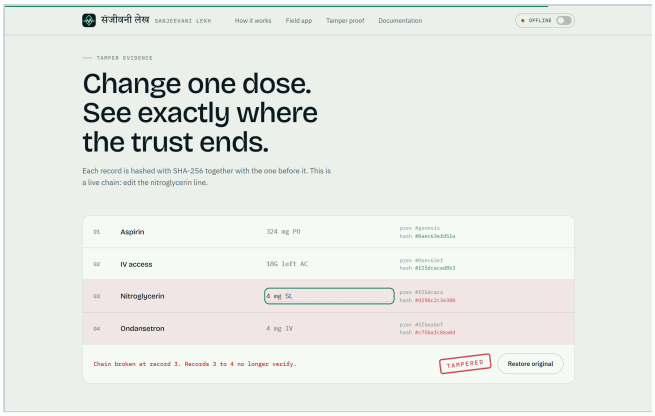
Three ways in: voice, typed text, photo of a note.



A scroll-driven timeline of one call, 14:32 to 14:51.

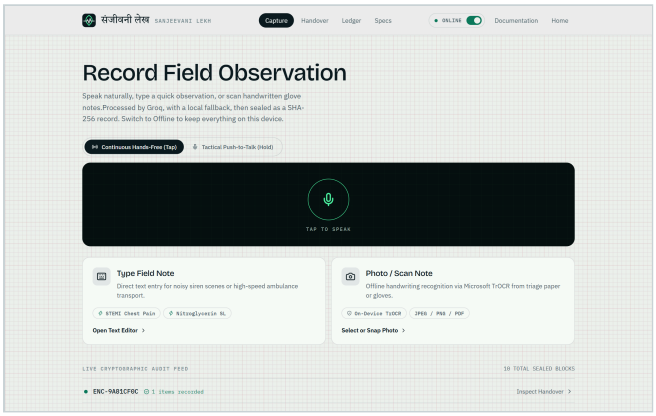


Offline demo: records queue on the device and flow to the cloud when the switch goes online.

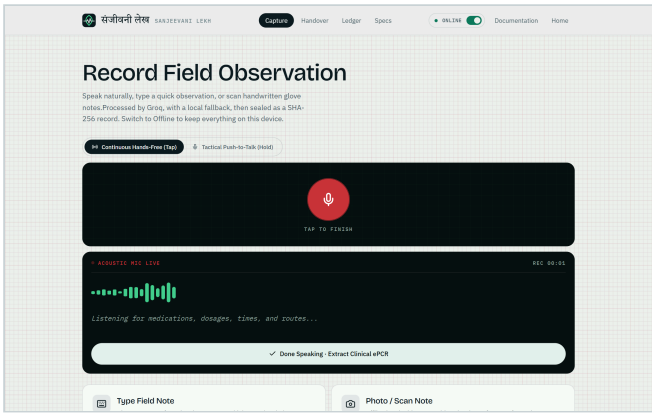


Tamper demo: a live SHA-256 chain; editing one dose breaks every later record.

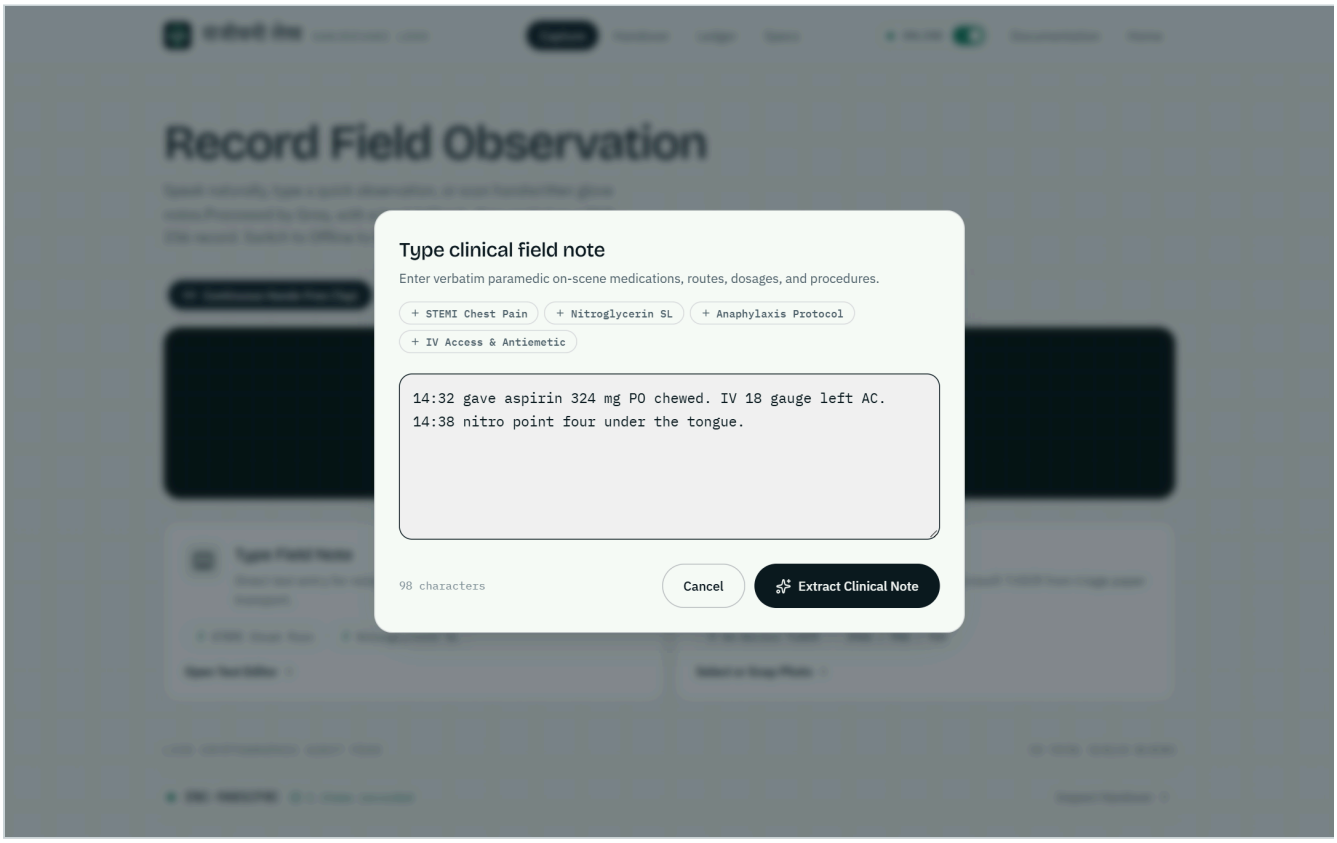
7.2 Field app: capture



Capture screen with the ONLINE / OFFLINE switch in the bar.



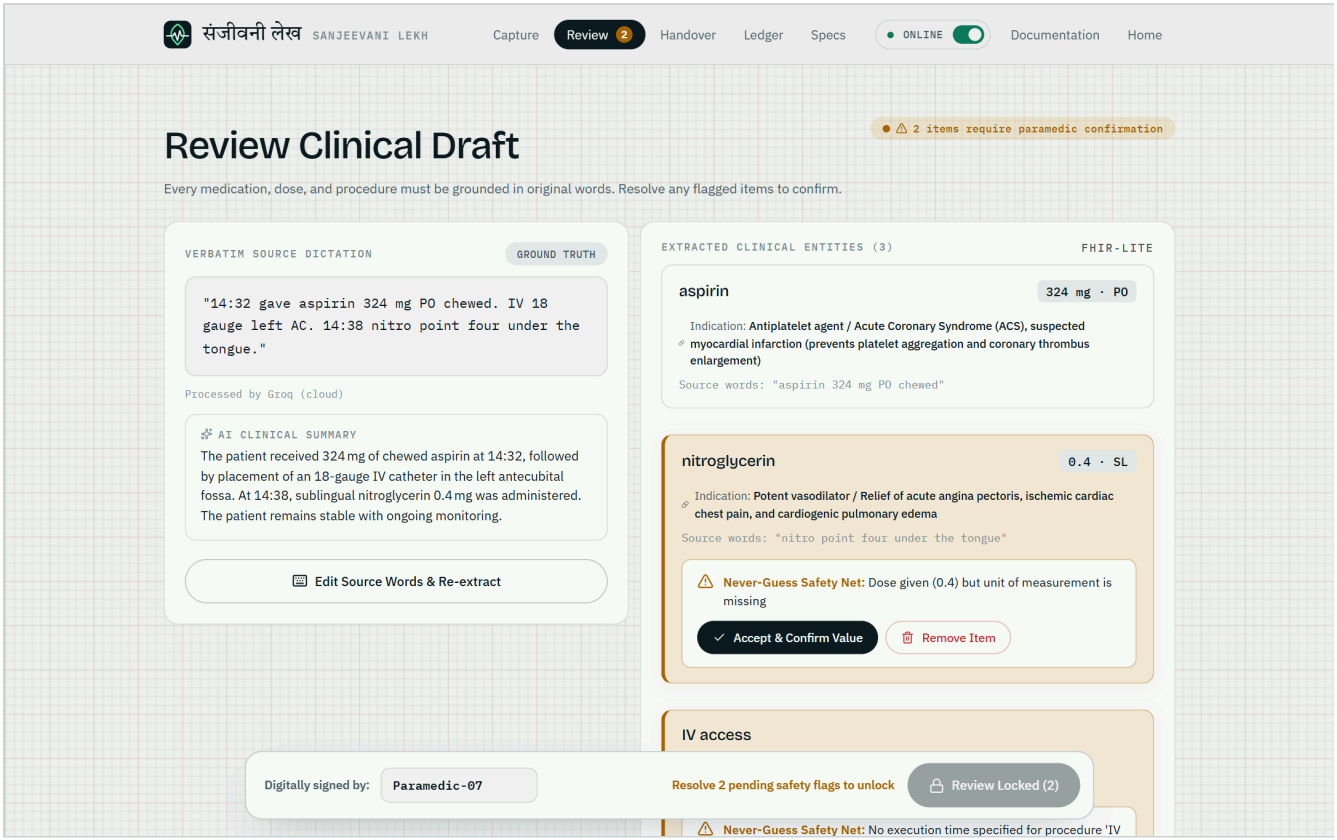
Recording: live waveform and transcript on a monitor-style panel.



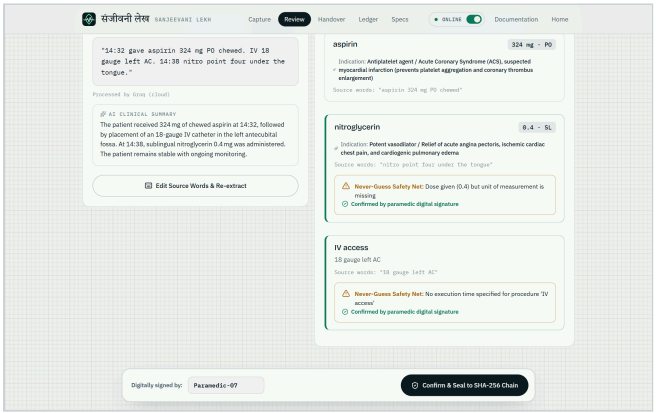
Typed entry with one-tap sample notes.

7.3 Field app: review, confirm, seal

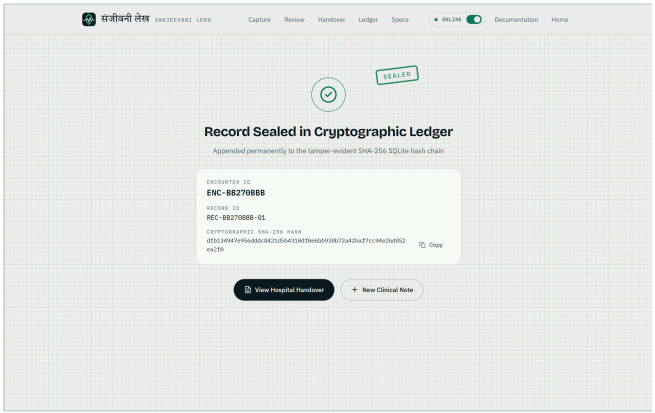
The review screen keeps the responder's original words on the left and the extracted items on the right. Each flagged item says why it was flagged. The Confirm button stays locked until every flag is accepted or the item removed. On sealing, the app shows the new record's hash.



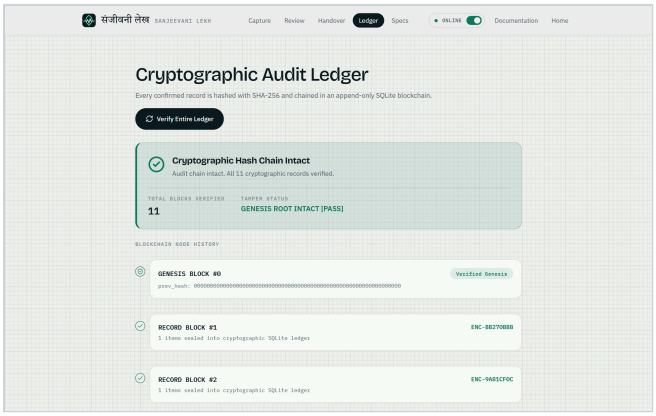
Review with two open flags: the nitroglycerin unit was not spoken, and the IV has no time.



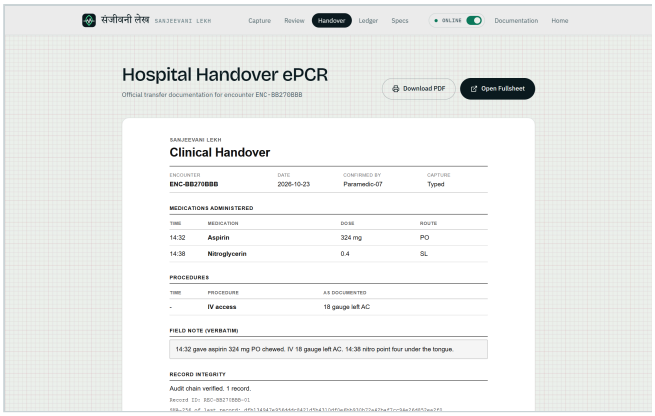
Flags resolved: Confirm is unlocked. The engine is shown under the source words.



Sealed: record ID, encounter ID and SHA-256 hash.



Ledger: the whole chain re-verified on demand.



Handover preview inside the app with PDF download.

7.4 Handover report (PDF)

The handover is a plain one-page form: encounter, date, who confirmed it and how it was captured; medications and procedures in time order; the verbatim field note; the integrity line with the record ID and full hash; and a sign-off box for the receiving clinician. It contains no machine-written narrative, only what is in the confirmed record.

SANJEEVANI LEKH

Clinical Handover

ENCOUNTER	DATE	CONFIRMED BY	CAPTURE
ENC-BB270BBB	2026-10-23	Paramedic-07	Typed

MEDICATIONS ADMINISTERED

TIME	MEDICATION	DOSE	ROUTE
14:32	**Aspirin**	324 mg	PO
14:38	**Nitroglycerin**	0.4	SL

PROCEDURES

TIME	PROCEDURE	AS DOCUMENTED
-	**IV access**	18 gauge left AC

FIELD NOTE (VERBATIM)

14:32 gave aspirin 324 mg PO chewed. IV 18 gauge left AC. 14:38 nitro point four under the tongue.

RECORD INTEGRITY

Audit chain verified. 1 record.

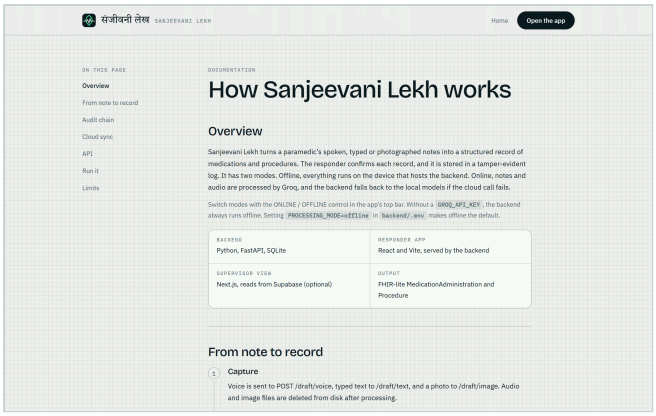
```
Record ID: REC-BB270BBB-01
SHA-256 of last record: dfb134947e956dddc8421d5b4310df0e6bb930b72a42baf7cc94e26d852ea2f0
Extraction: groq/openai/gpt-oss-120b
```

RECEIVED BY (NAME AND ROLE) SIGNATURE TIME

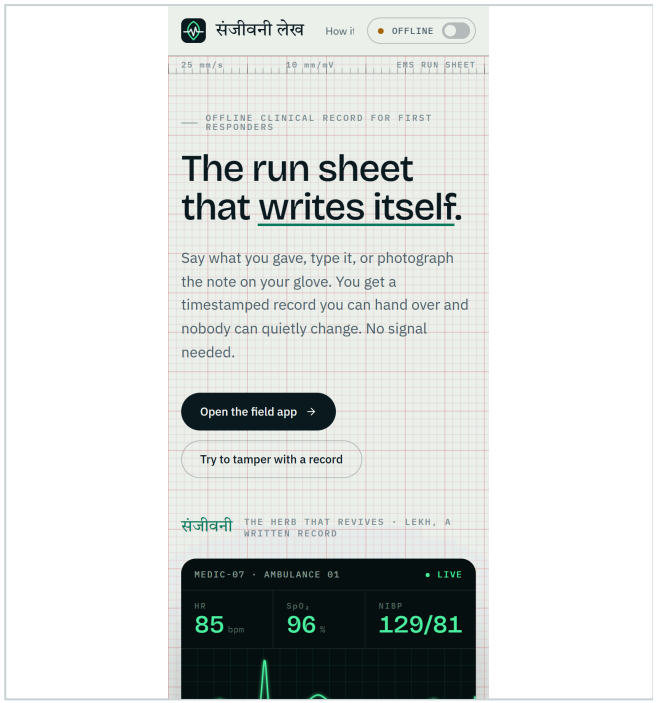
Prepared from field documentation confirmed by the responder named above. Not a certified medical record.

The generated PDF.

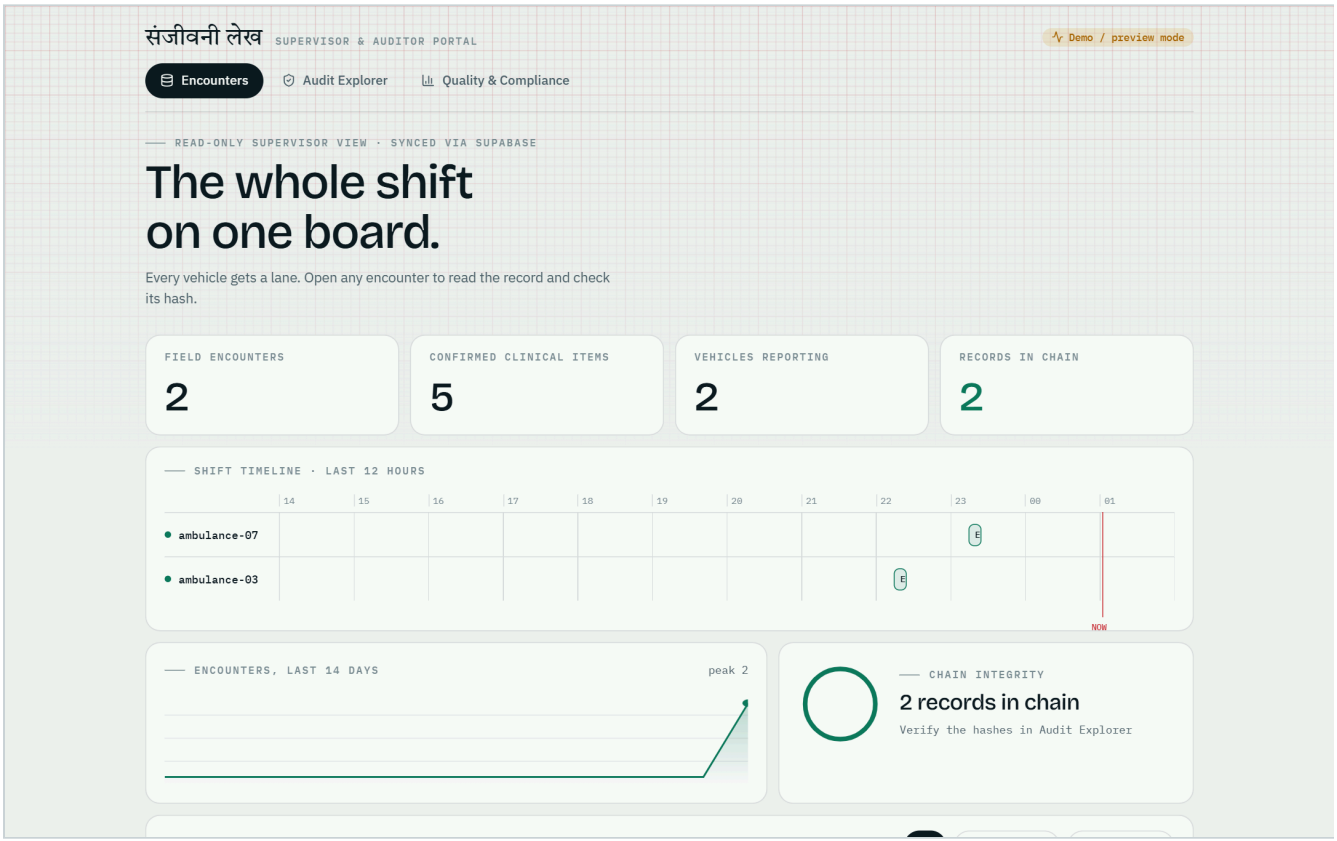
7.5 Documentation page and supervisor dashboard



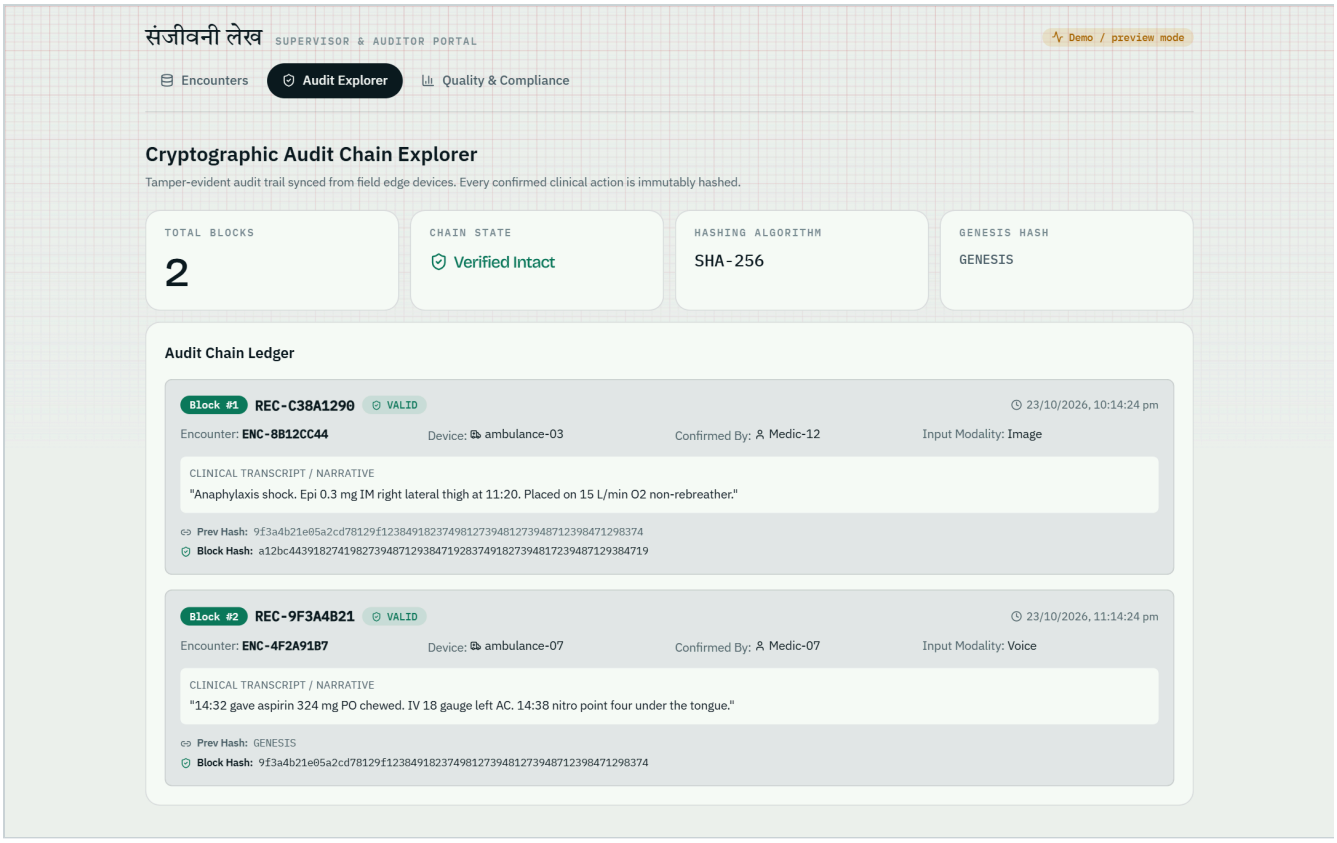
In-app documentation at `/docs`; the API explorer is at `/api/docs`.



The landing page at phone width.



Supervisor dashboard (sample data): counts, shift board by vehicle, 14-day sparkline, chain count and encounter table.



Audit explorer: every synced block with its previous hash and block hash.

8 API reference

All routes are served by the backend. Interactive documentation is at `/api/docs`. The draft routes (`text`, `voice`, `image` and `reextract`) accept the optional header `X-Processing-Mode: online | offline`.

METHOD	PATH	PURPOSE
GET	`/health`	Database, Ollama and model status; whether a Groq key is configured
POST	`/draft/text`	Typed note to draft
POST	`/draft/voice`	Audio file to draft
POST	`/draft/image`	Photo to draft
POST	`/draft/reextract`	Re-run extraction on edited text
POST	`/confirm`	Append a confirmed record to the chain
POST	`/correct/{record_id}`	Append a correction linked to a record
GET	`/audit/verify`	Re-check every hash in the chain
GET	`/encounters`	List encounters with record counts and sync status
GET	`/encounters/{id}`	All records for one encounter
GET	`/encounters/{id}/report`	Handover report as HTML
GET	`/encounters/{id}/report.pdf`	Handover report as PDF

POST	`/extract`	Extraction only, no validation
POST	`/sync`	Push unsynced records to Supabase
POST	`/clinical-ner/entities`	Treatments, problems and tests in a text
POST	`/clinical-ner/embedding`	Bio_ClinicalBERT embedding of a text

9 Testing and measured results

The backend has 94 automated tests across 14 files. In a clean install without the optional model downloads, **85 pass**, 1 is skipped, 1 (marked `llm`) needs a running Ollama model, and 7 fail only because they need models that are not installed: the six clinical-NER tests (n2c2 and Bio_ClinicalBERT weights) and one handwriting OCR test (TrOCR weights).

AREA	COVERED BY
Audit chain and triggers	`test_db.py`
Validation rules, normalisation	`test_validate.py`, `test_normalise.py`
Extraction, rule-based fallback	`test_extract.py`, `test_rule_extractor.py`
FHIR mapping	`test_fhir.py`
Draft routes, voice and photo routes	`test_draft_text.py`, `test_transcribe.py`, `test_ocr.py`
Online/offline routing and fallback (Groq mocked)	`test_cloud.py`
Reports	`test_report.py`
Sync	`test_sync.py`

Not covered: there are no automated browser tests for the interface; speech and handwriting accuracy have not been measured. The suite is forced offline by `tests/conftest.py`, so a developer's `.env` never makes a test call Groq.

10 Running the project

Requirements: Python 3, Node.js 20 or newer. For offline extraction, an Ollama server with the chosen model. Copy `backend/.env.example` to `backend/.env` and fill in the values you need.

```
cd backend
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000

cd local-ui
npm install
npm run build                    # the backend serves local-ui/dist at http://localhost:8000

cd dashboard                     # optional supervisor view
npm install
npm run dev
```

Optional model downloads for the clinical NER and TrOCR features are handled by `backend/scripts/download_models.py`. `backend/scripts/evaluate.py` runs the benchmark, and `backend/scripts/tamper_demo.py` demonstrates the chain detecting an edit.

Repository layout

```
backend/
  app/            FastAPI app and all modules
  prompts/        extraction, retry and whisper-hint prompts (versioned)
  data/           drug_reference.json
  templates/      report.html
  scripts/        download_models, evaluate, tamper_demo
  tests/          14 test files
local-ui/         React + Vite responder app, landing page, docs
dashboard/        Next.js supervisor dashboard + supabase_schema.sql
test-data/        20 synthetic notes + answer keys
DOCS/             architecture, design, requirements, prompts, this guide
```

11 Limitations and next steps

Limitations today

- Not a medical device and not clinically validated. A responder must confirm every record.
- The drug reference list has 17 drugs; anything else is flagged as unknown.
- Online mode sends text and audio to a third party. Offline extraction needs Ollama, and without it the built-in rules are noticeably less accurate (section 4.2).
- The on-screen live transcript uses the browser's speech recognition where available, and some browsers send that audio to the vendor. The saved record uses the backend's own transcription.
- The audit chain detects tampering but does not by itself prove when records were created, because timestamps come from the device clock.
- The dashboard is read-only and has no login of its own beyond the Supabase configuration.

Reasonable next steps (not built)

- Measure the local Ollama model and Whisper on the same benchmark, and grow the benchmark beyond 20 notes.
- Anchor the latest hash outside the device, for example on the handover form or in the cloud copy, and verify it on the dashboard.
- Add automated browser tests for the review-and-confirm flow.
- Broaden the drug reference and add configurable local protocols.